

RA3D Software\PrintController.py

```

1  '''Print controller. Executes gcode commands. Contains main print loop, and bed leveling related
   commands
2  such as bed calibration and corner sweep'''
3  from tkinter import *
4  from tkinter import filedialog
5  import os, re, copy, threading, time, math
6  import numpy as np
7  from ArmController import Position, Origin, MoveCommand, MoveParameters
8
9  class PrintController:
10     #region init
11     def __init__(self, root, armController):
12         self.root = root
13         self.armController = armController
14         #-----Important variables-----
15         #MoveParameters(speed, acceleration, deceleration, ramp, loopmode, speedtype)
16         self.speed = 50 #mm/s
17         self.acceleration = 5 #percent of move that is accelerating
18         self.decceleration = 5 #percent of move that is decelerating
19         self.ramp = 20 #minimum is 10
20         #0=closed loop
21         self.printOpenLoopControl = 0 #Thus closed loop
22         self.defaultPrintParameters =
           MoveParameters(self.speed, self.acceleration, self.decceleration, self.ramp, self.printOpenLoopControl, "m")
           #50mm/s print speed
23         self.timeoutExtra = 60 #Extra timeout added to every timeout estimation
24         self.ignoreFlags = False #ignores flags and unrecognized gcode lines and boundary check
25         self.checkBoundaryTrue = True #enables/disable boundary checks
26
27         self.plateHeight = 265 #Change bed height relative to pen
28         self.dropHeight = 0.0 #mm, drop all layers by amount, z will not go negative
29         #boundaries for corner calibration/setting recommended origin
30         #These boundaries are for better printing
31         self.YEdge = [-50, 50] #These values were adjusted carefully, should not change
32         self.XEdge = [400, 500]
33
34         self.hardCodePrinterSpeed = False #Will obey the print parameters and not the gcode feed rate
35         self.axis5 = False #NOTE should be set to false because axis 5 implementation is incomplete
36
37         #Boundaries to stop printing
38         #These boundaries are to prevent dangerous movement
39         self.maxBoundaryX = [270, 530]
40         self.maxBoundaryY = [-130, 130]
41         self.maxBoundaryZ = [200, 900]
42         self.bufferBoundary = 10 # Warning pops up if this boundary is entered from the max boundaries
43         self.bedCalibrateHeight = 50 #Height moved up for calibration
44
45
46         self.maxHotEndTempSetting = 300 #Max hotend temp that can be set from gcode
47         self.maxBedTempSetting = 120 #Max bed temp that can be set from gcode
48
49         self.backlashAngleOffset = 0 #5.55 #offset to account for backlash

```

```

50         #Could add another safeguard within the temperatur controller
51         #-----End important variables-----
52
53         #recomended Origin for move set at middle of calibration corners
54         self.recommendedOriginPosition =
Position((self.XEdge[0]+self.XEdge[1])/2,0,self.plateHeight,0,90,0,None)
55         #extract position to origin
56         self.recommendedOrigin = self.recommendedOriginPosition.toOrigin()
57
58         #Assume origin is at recommended origin
59         self.origin = self.recommendedOrigin
60         self.defaultAngle = 90-self.backlashAngleOffset
61         #self.defaultAngle = self.calculateBacklashOffset((self.XEdge[0]+self.XEdge[1])/2)
62         self.recommendedOriginPosition.Ry = self.defaultAngle
63
64         # Calibration corners
65         FLCorner = Position(self.XEdge[1],self.YEdge[0],self.plateHeight,0,self.calculateBacklashOff-
set(self.XEdge[1]),0,None)
66         FRCorner = Position(self.XEdge[1],self.YEdge[1],self.plateHeight,0,self.calculateBacklashOff-
set(self.XEdge[1]),0,None)
67         BLCorner = Position(self.XEdge[0],self.YEdge[0],self.plateHeight,0,self.calculateBacklashOff-
set(self.XEdge[0]),0,None)
68         BRCorner = Position(self.XEdge[0],self.YEdge[1],self.plateHeight,0,self.calculateBacklashOff-
set(self.XEdge[0]),0,None)
69
70         #corners are absolute relative to the physical structure
71         #but z values of calibration will update to be the same as origin if the origin is changed in
the UI
72         #however calibration corners z remain unchanged after initialization
73         self.calibrationCorners = [FRCorner,BRCorner,BLCorner,FLCorner]
74
75         #----- Setup variables-----
76         # Variables that are just simply initialized
77
78         # Parameters used for saving the last used coordinate information
79         self.lastPos = Position(None,None,None,None,None,None,self.origin)
80         self.printPos = Position(None,None,None,None,None,None,self.origin)
81
82         self.lastF = 0.0
83         self.lastE = 0.0
84         self.feedRate = 0
85         self.extrudeRate = 0
86         self.currentInstruction = 0
87
88         self.justMovedHome = False
89         self.relativeExtrusion = True
90         self.relativePositioning = False #default is abosolute positioning
91
92         self.selectedFilepath = None
93         self.gcodeLines = []
94         self.teensyLines = []
95         self.fileOpen = False
96         self.printing = False
97         self.printPaused = False
98         self.cornerSweeping = False

```

```

99     self.bedCalibrationInProgress = False
100    self.bedCalStep = 0
101
102    #Flag variable for errors
103    self.flag = None
104
105
106
107    #endregion init
108    #region ===== Main Functions =====
109
110    # This is the main function that will loop when printing a file
111    def printLoop(self):
112        # NOTE for proper flag handling the program should repeat the last command depending on the
flag, this is not been implemented
113        self.checkFlag() #check flag to see if to continue printing
114
115        #If the flag stopped the print, exit
116        if self.printing == False or self.printPaused == True:
117            self.root.printThreadStarted = False
118            return
119
120        message = "comment"
121        #G code lines will skip past comments
122        while message == "comment":
123            #check if at end of file
124            if self.currentInstruction > len(self.gcodeLines) - 1:
125                self.root.statusPrint("End of program reached")
126                self.endPrint() #Do any necessary processes to end the print
127                message = ""
128                break
129
130            # Pull current line
131            lineToConvert = self.gcodeLines[self.currentInstruction]
132
133            #Update progress bar
134            self.currentInstruction += 1 # Increment currentInstruction
135            self.root.progressBarHome['value'] = (self.currentInstruction / len(self.gcodeLines))
* 100 # Update home progress bar to match
136            self.root.progressHomeLabel.config(text=f"{round((self.currentInstruction /
len(self.gcodeLines)) * 100, 2)}%")
137
138            #Read gcode line and convert, handle rare messages inside
139            message = self.interpretGcode(lineToConvert) # Convert line and updates printPos
140
141            #Display gcode lines for debugging and if a lineToConvert exists
142            if lineToConvert and self.root.PrintDebugMode:
143                self.root.terminalPrint(f"Line: {lineToConvert}")# Print the line we're converting
144
145            #region -----Message Processing-----
146            #Common commands are handled here, rare commands handled inside interpretGcode
147            if message == "":
148                pass
149            elif message[:5] == "Error": # If the point is blank, don't try to send a command
150                self.root.warningPrint("Error printing "+ message)

```

```

151         if not self.ignoreFlags:
152             self.pausePrint()
153     # TODO: Add ability to home one axis
154     elif message[:4] == "Home":
155         self.root.armController.moveHome()
156         #TODO check to see if home printer only happens at beginning of gcode for 5 axis printing
157         #When we combine 2 gcodes the 2nd one cannot move origin
158         self.justMovedHome = True #For better wrist condition after moving from home position
159     elif message == "Success":
160         #TODO when turn hazard encountered,
161         # printer moves home than to position with normal wrist condition
162         #Execute move command
163         moveParameters = copy.deepcopy(self.defaultPrintParameters)
164         if not self.hardCodePrinterSpeed:
165             if self.feedRate != None and self.feedRate != 0:
166                 moveParameters.speedType = "m"
167                 #Conver to mm/s from mm/min
168                 moveParameters.speed = self.feedRate / 60
169         #estimate timeout
170         timeout =
self.root.armController.estimateMoveTime(self.lastPos,self.printPos,moveParameters.speed)
171         timeout += self.timeoutExtra #extra time for communication
172
173         #if valid position and no flag
174         if self.printPos.x is not None and self.flag is None:
175             if self.root.PrintDebugMode:
176                 self.root.terminalPrint(f"Point: {self.printPos.GetAbsolute()}") # Print the
returned point list
177
178                 #So that when coming from a home command it moves with optimal wrist condition
179                 if self.justMovedHome:
180                     moveParameters.wrist = "N"
181                     self.justMovedHome = False
182                     self.root.armController.sendMJ(self.printPos, moveParameters=moveParameters,
timeout=timeout)
183                 #if there is not extrusion, movement doesn't have to be a straight line
184                 elif self.extrudeRate == 0:
185                     moveParameters.wrist = "N"
186                     #self.root.armController.moveHome()#NOTE this line is for drawing tests only
187                     self.root.armController.sendMJ(self.printPos, moveParameters=moveParameters,
timeout=timeout)
188                 else:
189                     #Check if hotend is at target temperature before extruding, if not set flag to stop
print and display warning
190                     if not self.root.temperatureController.HotendTargetReached():
191                         self.flag = "Hotend not at target temperature"
192                     else:
193                         # Send the command to the arm, will wait for a response
194                         self.root.armController.sendML(self.printPos, moveParameters=moveParameters,
extrudeRate=self.extrudeRate,timeout=timeout, RelativeExtrude = self.relativeExtrusion)
195
196         elif message == "Extrusion Only":
197             self.root.terminalPrint(f"Extruding {self.extrudeRate} without moving")
198             #Must be at hotened before extruding
199             if not self.root.temperatureController.HotendTargetReached():

```

```

200         self.flag = "Hotend not at target temperature"
201     #If a feedrate was spec
202     if self.feedRate != None and self.feedRate != 0:
203         moveParameters = copy.deepcopy(self.defaultPrintParameters)
204         moveParameters.speedType = "m"
205         #Conver to mm/s from mm/min
206         moveParameters.speed = self.feedRate / 60
207     else:
208         #Use default extrusion parameters rather default print parameters
209         moveParameters = copy.deepcopy(self.defaultPrintParameters)
210     if self.flag is None:
211
self.root.armController.extrude(self.extrudeRate,moveParameters=moveParameters,timeoutMultiplier=2,
RelativeExtrude = self.relativeExtrusion)
212     else:
213         self.root.warningPrint("Unexpected message from interpretGcode")
214     #endregion message handling
215
216     #must be last thing to do, copy last position
217     self.lastPos = copy.deepcopy(self.printPos)
218
219     #Signifies thread has ended to start next thread
220     self.root.printThreadStarted = False #Do NOT return anywhere else in this function
221
222     # Converts a GCode instruction to the instruction to send over serial
223     # Manages rare gcode commands and updates the print position based on the gcode command
224     def interpretGcode(self, lineToConvert):
225         #Assume extruderate will not be set
226         self.extrudeRate = 0
227
228         #NOTE some of these gcode commands are a work in progress
229         if lineToConvert[0] == ';': # Line is comment
230             return "comment" # Don't convert
231         elif lineToConvert[0:3] == "G21":
232             return ""
233         elif lineToConvert == "\n": # Line is newline
234             return "" # Don't convert
235         #Relative Extrusion
236         elif lineToConvert[:3] == "M83":
237             self.root.terminalPrint("Using relative extrusion")
238             self.relativeExtrusion = True
239             return ""
240         #Absolute Extrusion
241         elif lineToConvert[:3] == "M82":
242             self.root.terminalPrint("Using absolute extrusion")
243             self.relativeExtrusion = False
244             return ""
245
246         #Temperature control command
247         #Set Hot End Temperature
248         elif lineToConvert[:4] == "M104":
249             self.root.terminalPrint("Setting hotend temperature...")
250             sMatch = re.search(r"[sS](-?(?:\d+\.\d*|\.\d+))", lineToConvert)
251             s = float(sMatch.group(1)) if sMatch else None
252             if s is not None and s >= 0 and s <= self.maxHotEndTempSetting:

```

```

253         self.root.temperatureController.setHotendTargetTemp(s)
254         self.root.temperatureController.enableHotendControl()
255     return ""
256
257 #Set Bed Temperature
258 elif lineToConvert[:4] == "M140":
259     self.root.terminalPrint("Setting bed temperature...")
260     sMatch = re.search(r"[sS](-?(?:\d+\.\d*|\.\d+))", lineToConvert)
261     s = float(sMatch.group(1)) if sMatch else None
262     if s is not None and s >= 0 and s <= self.maxBedTempSetting:
263         self.root.temperatureController.setBedTargetTemp(s)
264         self.root.temperatureController.enableBedControl()
265     return ""
266
267 #Wait for Hot End Temperature
268 elif lineToConvert[:4] == "M109":
269     self.root.terminalPrint("Waiting for hotend to reach target temperature...")
270     while not self.root.temperatureController.HotendTargetReached():
271         time.sleep(1) # Wait for 1 second before checking again
272     self.root.terminalPrint("Hotend reached target temperature")
273     return ""
274
275 #Wait for Bed Temperature
276 elif lineToConvert[:4] == "M190":
277     self.root.terminalPrint("Waiting for bed to reach target temperature...")
278     while not self.root.temperatureController.BedTargetReached():
279         time.sleep(1) # Wait for 1 second before checking again
280     self.root.terminalPrint("Bed reached target temperature")
281     return ""
282
283 elif lineToConvert[:3] == "G28": # Home the printer
284     return "Home" + lineToConvert[3:]
285 elif lineToConvert[:3] == "G90": # Absolute positioning
286     self.relativePositioning = False
287     self.root.terminalPrint("Using absolute positioning")
288     return ""
289 #relative positioning
290 elif lineToConvert[:3] == "G91":
291     self.relativePositioning = True
292     self.root.terminalPrint("Using relative positioning")
293     return ""
294
295 #Zero extruder axis
296 elif lineToConvert == "G92 E0\n":
297     self.root.armController.zeroJ7()
298 #wait
299 elif lineToConvert[:2] == "G4":
300     span = float(lineToConvert[3:])
301     time.sleep(span/1000)
302 elif lineToConvert[0:2] == "G0" or lineToConvert[0:2] == "G1": # Move (treating G0 & G1 as
mostly equal)
303
304     #Search for values
305     xMatch = re.search(r"[xX](-?(?:\d+\.\d*|\.\d+))", lineToConvert)

```

```

306 yMatch = re.search(r"[yY](-?(?:\d+\.\d*|\.\d+))", lineToConvert)
307 zMatch = re.search(r"[zZ](-?(?:\d+\.\d*|\.\d+))", lineToConvert)
308 fMatch = re.search(r"[fF](-?(?:\d+\.\d*|\.\d+))", lineToConvert)
309 eMatch = re.search(r"[eE](-?(?:\d+\.\d*|\.\d+))", lineToConvert)
310
311 #For 5 axis printing
312 if self.axis5:
313     uMatch = re.search(r"[uU](-?(?:\d+\.\d*|\.\d+))", lineToConvert)
314     vMatch = re.search(r"[vV](-?(?:\d+\.\d*|\.\d+))", lineToConvert)
315     u = float(uMatch.group(1)) if uMatch else None
316     v = float(vMatch.group(1)) if vMatch else None
317 else:
318     u = None
319     v = None
320
321 #Coordinates here are RELATIVE to the origin
322 #Absolute positiong
323
324 x = float(xMatch.group(1)) if xMatch else None
325 y = float(yMatch.group(1)) if yMatch else None
326 z = float(zMatch.group(1)) if zMatch else None
327
328 if z is not None:
329     #Adjust for change
330     z = z - self.dropHeight
331     z = max(z,0) #don't allow negative z values
332
333 #Relative positioning
334 if self.relativePositioning:
335     if x is not None:
336         x += self.lastPos.GetRelativeX()
337     if y is not None:
338         y += self.lastPos.GetRelativeY()
339     if z is not None:
340         z += self.lastPos.GetRelativeZ()
341
342 #print("z read:", z)
343 f = float(fMatch.group(1)) if fMatch else None
344 e = float(eMatch.group(1)) if eMatch else None
345
346 #NOTE 5 axis g code is not fully implemented
347 if self.axis5:
348     #For now both u and v are need for rotation
349     if u == None or v == None:
350         Rx = self.lastPos.Rx
351         Ry = self.lastPos.Ry
352         Rz = self.lastPos.Rz
353     else:
354         u += -90 #may add instead of subtract
355         #TODO I think u is elevation and v azimuth, need to check
356         Rz,Ry,Rx = self.aer_to_euler_zyx(v,u,0) # 3 options for transformation
357
358 NoMove = False #For some commands they only specify E
359

```

```

360         if x == None and y==None and z==None:
361             NoMove = True
362             self.printPos=copy.deepcopy(self.lastPos)
363         else:
364             # If GCode instruction didn't contain a parameter, pull from last saved value
365             # If instruction DID contain a parameter, offset the value to put it in the build
volume
366             if x == None:
367                 x = self.lastPos.GetRelativeX()
368             if y == None:
369                 y = self.lastPos.GetRelativeY()
370             if z == None:
371                 z = self.lastPos.GetRelativeZ()
372             #if using 5 axis g code
373             if self.axis5:
374                 self.printPos.SetRelative(x,y,z,Rx,Ry,Rz)
375             else:
376                 self.printPos.SetRelative(x,y,z,0,self.calculateBacklashOffset(x,relative=True),0)
377
378             #Do not extrude if not told to
379             #Handle If E or F or both are missing
380             if not NoMove:
381                 if e == None:
382                     e = 0
383                 else:
384                     self.lastE = e
385             elif e == None and f==None:
386                 #extrusion has to be specified for extrusion only
387                 return "Error no movement specified"
388             elif e == None:
389                 self.feedRate = f
390                 return ""
391             elif f == None:
392                 self.extrudeRate = e
393                 return "Extrusion only"
394
395             atBoundary = False
396             #If values are past max boundaries set to be at boundaries
397             if self.checkBoundaryTrue:
398                 if self.printPos.y > self.YEdge[1]:
399                     atBoundary = True
400                 if self.printPos.y < self.YEdge[0]:
401                     atBoundary = True
402                 if self.printPos.x > self.XEdge[1]:
403                     atBoundary = True
404                 if self.printPos.x < self.XEdge[0]:
405                     atBoundary = True
406                 if atBoundary:
407                     self.root.terminalPrint("Warning moving/printing near boundaries, print will
continue")
408
409             #G0 moves printer at max speed
410             if lineToConvert[0:2] == "G0":
411                 self.feedRate = 5000#mm/min
412             else:

```



```

413         if f is not None:
414             self.feedRate = f
415
416         #Set extrude rate of printer
417         self.extrudeRate = e
418
419         #One last check if point is in boundary else pause print
420         if self.checkBoundary(self.printPos):
421             return "Success"
422         else:
423             #NOTE this disabled with ignore flags
424             if not self.checkBoundaryTrue:
425                 self.pausePrint()
426                 self.root.warningPrint("Moving of bounds")
427                 return "Error moving out of bounds"
428     else:
429         pass
430         #Currently most gcode commands are implemented
431         #Uncomment this line if you need this safeguard
432         #if not self.ignoreFlags:
433             # return "Error unrecognized gcode line"
434
435     #Return nothing
436     return ""
437
438 #Function used when end of print is reached or when print is cancelled to end any related processes
439 def endPrint(self, moveHome = True):
440     self.root.temperatureController.disableHotendControl()
441     self.root.temperatureController.disableBedControl()
442     self.root.LEDOn = False # Turn off LED to signify print is cancelled
443     self.currentInstruction=0
444     self.printing = False
445     self.printPaused = False #If print was paused, it is no longer paused if it is cancelled or
ended
446     #Move Home when complete
447     self.root.armController.moveHome()
448
449     #endregion main functions
450
451     #region =====| GUI Buttons |=====
452     #These functions will relate to the buttons in the GUI
453     #TODO add busy check for any user implementation and add more flags if needed
454     def selectFile(self,userSelect = True, filepath = None):
455         # TODO: Needs some form of garbage collection as Python holds onto the memory allocated when
opening a file
456         # TODO: One possibility is utilizing "yield" command (or other methods) to only read one line
at a time
457
458         # This list contains valid file types
459         filetypes = [
460             ("GCode Files", "*.gcode"),
461             ("All Files", ".*.*")
462         ]
463         # Have user select a file
464         if userSelect:

```

```
465         self.selectedFilepath = filedialog.askopenfilename(filetypes=filetypes)
466     else:
467         self.selectedFilepath = filepath
468
469     # Check if user actually selected a file
470     if (self.selectedFilepath == ""):
471         self.root.statusPrint("No file selected")
472         self.root.selectedFileHomeLabel.config(text="Please select a file")
473         self.gcodeLines = [""]
474         self.fileOpen = False
475         # Disable the buttons
476         self.root.startPrintHomeButton.config(state="disabled")
477         self.root.pausePrintHomeButton.config(state="disabled")
478         self.root.stopPrintHomeButton.config(state="disabled")
479         return
480     # Change selectedFileLabel to have filename
481     self.root.statusPrint(f"Selected \"{os.path.basename(self.selectedFilepath)}\"")
482     self.root.selectedFileHomeLabel.config(text=os.path.basename(self.selectedFilepath))
483     self.fileOpen = True
484     self.currentInstruction = 0 # Reset the currentInstruction counter
485
486     # Read all lines of the file into gcodeLines
487     selectedFile = open(self.selectedFilepath, "r")
488     self.gcodeLines = selectedFile.readlines()
489     selectedFile.close()
490
491     # Enable the buttons
492     self.root.startPrintHomeButton.config(state="normal")
493     self.root.pausePrintHomeButton.config(state="normal")
494     self.root.stopPrintHomeButton.config(state="normal")
495
496     def autoSelectFile(self, filepath):
497         try:
498             self.selectFile(userSelect=False, filepath=filepath)
499             self.root.terminalPrint(f"Automatically selected file: {filepath}")
500         except:
501             self.root.terminalPrint(f"Failed to automatically select file: {filepath}")
502
503     #Start the print
504     def startPrint(self):
505
506         self.syncOrigin()#Get origin from arm controller
507
508         #Check for problems that would prevent print from starting
509         if not self.origin.checkOriginSet():
510             self.root.statusPrint("Origin not set, print cancelled")
511             return
512         if self.root.armController.checkIfAllBusy(message="start print"):
513             return
514         #Warning prints
515         if self.ignoreFlags:
516             self.root.warningPrint("Ignore flags is enabled\nFlags will be ignored. Disable in print
controller")
517         if not self.checkBoundaryTrue:
```

```
518         self.root.warningPrint("Boundary check disabled, arm may move dangerously")
519     if self.root.coolendMode:
520         self.root.warningPrint("Printing in coolend mode. Extruder should not be attached to the
hotend else damage will occur.")
521
522     #Print starts here if no problems
523     self.root.LEDOn = True # Solid LED to signify print in progress, will be turned off when print
is paused or finished
524
525     #Resume print if paused
526     if self.printPaused == True and self.printing == True:
527         self.printPaused = False
528         return #all other initialization ignored when resuming print
529
530     self.currentInstruction = 0 # Reset the currentInstruction counter to start of file
531
532     #Zero extruder axis used for measuring total filament used
533     self.root.armController.zeroJ7()
534
535     #Reset flag
536     self.flag = None
537     self.root.serialController.clearQueue() #clear queue so it is not backed up
538
539     self.printPos.origin = self.origin
540     self.printPos = self.origin.toPosition(angle=self.calculateBacklashOffset(self.origin.x))
#Reset print position to origin
541     #Last position starts at origin
542     self.lastPos = Position(self.origin.x,self.origin.y,self.origin.z,0,self.calculateBacklashOff-
set(self.origin.x),0,self.origin)
543
544     #Reset feedrate and extruderate
545     self.lastF = 0.0
546     self.lastE = 0.0
547     self.printing = True
548     self.root.statusPrint("Starting print...")
549     self.root.printStatusHomeLabel.config(text="PRINTING...")
550
551     #Use to step the print one line at a time
552     def stepPrint(self):
553         if self.checkIfPrinterBusy():
554             self.root.terminalPrint("Cannot step print, printer busy")
555         if self.root.armController.checkIfBusy():
556             self.root.terminalPrint("Cannot step print, arm busy")
557         #will perform one print loop only
558         self.printLoop()
559
560     #Pause the print
561     def pausePrint(self):
562         # Check if we are actually printing
563         if self.printing:
564             self.root.LEDOn = False # Turn off LED to signify print is paused
565             self.root.terminalPrint("Pausing Print")
566             self.root.printStatusHomeLabel.config(text="PAUSED...")
567             self.printPaused = True
568
```

```

569     #Cancel the print
570     def cancelPrint(self,moveHome = True):
571         self.endPrint(moveHome=moveHome) #Do any necessary processes to end the print
572         self.root.statusPrint("Print cancelled")
573         self.root.printStatusHomeLabel.config(text="IDLE...")
574         pass
575
576     # Bed Calibration and sweeps =====
577
578     #Start the print bed calibration
579     def startPrintBedCalibration(self):
580         self.flag = None #set flag to none only on start
581         #If anything is busy cannot start arm
582         if self.root.armController.checkIfAllBusy():
583             self.root.statusPrint("Printer is busy, cannot start bed calibration")
584             return
585         self.root.serialController.clearQueue() #clear queue so it is not backed up
586         self.bedCalibrationInProgress = True #flag to signal calibration
587         self.bedCalStep = 1 #Start at 1st corner
588
589         self.syncOrigin() #Sync origin with arm controller
590         #Update the corner z so that match a new origin
591         #The x and y stay fixed so origin can be offset from center.
592         self.plateHeight = self.origin.z
593         for corner in self.calibrationCorners:
594             corner.z = self.origin.z
595         #Move to next step (1)
596         self.nextBedCalibration()
597
598     #user controlled next button
599     def nextBedCalibration(self):
600         #Do not check all busy because bed calibration is one of the flags
601         if self.root.armController.checkIfBusy():
602             self.root.statusPrint("Arm is busy cannot go to next step")
603             return
604         if self.bedCalibrationInProgress==False:
605             self.root.statusPrint("Bed calibration not started")
606             self.checkFlag()
607         if self.bedCalibrationInProgress == True:
608             bedCalibrationThread = threading.Thread(target=self.bedCalibrationStep, name= "Bed
Calibration Thread")
609             bedCalibrationThread.start()
610
611     #start the corner sweep
612     def startCornerSweep(self):
613         self.flag = None #reset flag
614
615         if self.root.armController.checkIfAllBusy():
616             self.root.statusPrint("Arm is busy, cannot start corner sweep")
617             return
618         else:
619             self.cornerSweeping = True
620             self.root.serialController.clearQueue() #clear queue so it is not backed up
621             cornerSweepThread = threading.Thread(target=self.cornerSweep, name="Corner Sweep Thread")

```

```

622         cornerSweepThread.start()
623
624     #corner sweep and move up one level at a time
625     def startFullCornerSweep(self):
626         self.flag = None
627         if self.root.armController.checkIfAllBusy():
628             self.root.statusPrint("Arm is busy, cannot start corner sweep")
629             return
630         else:
631             self.cornerSweeping = True
632             self.root.serialController.clearQueue() #clear queue so it is not backed up
633             cornerSweepThread = threading.Thread(target=self.fullCornerSweep, name="Full Corner Sweep
Thread")
634             cornerSweepThread.start()
635
636     #will cancel any related printing setup functions
637     def cancelAny(self):
638         #just in case someone thinks this will cancel the print
639         if self.printing:
640             self.cancelPrint()
641         #Start on thread
642         threading.Thread(target=self.endSweepOrCal, name="End Sweep or Calibration Thread").start() #On
thread because move command is on there
643
644     #endregion gui
645
646     #region =====| Other Functions |=====
647     #All flag management should be done here besides raising a flag
648     def checkFlag(self):
649         #Assuming that pausing print is desired if there is a flag
650         flag = self.flag
651         if flag is not None and not self.ignoreFlags:
652             if self.printing:
653                 self.pausePrint()
654                 self.root.warningPrint(f"Print paused due to error: {self.flag}")
655                 self.flag = None
656             if self.bedCalibrationInProgress or self.cornerSweeping:
657                 self.endSweepOrCal()
658                 self.root.warningPrint(f"Bed calibration/corner sweeping paused due to error:
{self.flag}")
659                 self.flag = None
660             #Clear queue because queue tends to get clogged when encountering flags
661             self.root.serialController.clearQueue()
662             return flag
663         elif self.flag is not None:
664             #display warning print instead of pausing when ignore flag is on
665             self.root.warningPrint("Flag: "+self.flag)
666             self.flag = None
667             #Clear queue because queue tends to get clogged when encountering flags
668             self.root.serialController.clearQueue()
669             return flag
670         return None
671
672     #Check if the printer is busy with printing, calibration or sweeping
673     def checkIfPrinterBusy(self,message = None,display=True):

```

```

674         if self.cornerSweeping or self.bedCalibrationInProgress or (self.printing and not
self.printPaused):
675             #if there is a message to display
676             if message and display:
677                 self.root.terminalPrint("Cannot" + message + ", Printer busy")
678             elif display:
679                 self.root.terminalPrint("Printer busy")
680             return True
681
682         return False
683
684     #Set origin to arm controller origin
685     def syncOrigin(self):
686         self.origin = self.root.armController.origin
687
688     #change recommened origin based on plate height
689     def changeRecommendedOrigin(self, PlateHeight):
690         self.recommendedOriginPosition.z = PlateHeight
691         #extract position to origin
692         self.recommendedOrigin = self.recommendedOriginPosition.toOrigin()
693         self.root.terminalPrint(f"Recommended origin set to {self.recommendedOrigin.getOrigin()}")
694
695     #determines whether a position is within the boundaries of the printer
696     def checkBoundary(self, pos):
697         # check whether position is within buffer boundary, if so print warning
698         if pos.x < self.maxBoundaryX[0] or pos.x > self.maxBoundaryX[1]:
699             self.root.terminalPrint("X boundary exceeded")
700             print(pos.x < self.maxBoundaryX[0] + self.bufferBoundary or pos.x > self.maxBoundaryX[1] -
self.bufferBoundary)
701             self.root.terminalPrint(f"X at {pos.x} when exceeded")
702             if pos.y < self.maxBoundaryY[0] + self.bufferBoundary or pos.y > self.maxBoundaryY[1] -
self.bufferBoundary:
703                 self.root.terminalPrint("Y boundary exceeded")
704                 self.root.terminalPrint(f"Y at {pos.y} when exceeded")
705             if pos.z < self.maxBoundaryZ[0] or pos.z > self.maxBoundaryZ[1] :
706                 self.root.terminalPrint("Z boundary exceeded")
707                 self.root.terminalPrint(f"Z at {pos.z} when exceeded")
708             # Check if position is out of bounds, if so return false to prevent move
709             if pos.x < self.maxBoundaryX[0] or pos.x > self.maxBoundaryX[1] or pos.y <
self.maxBoundaryY[0] or pos.y > self.maxBoundaryY[1] or pos.z < self.maxBoundaryZ[0] or pos.z >
self.maxBoundaryZ[1]:
710                 return False
711             return True
712
713     #Caclulate pitch offset based on backlash using a linear equation
714     def calculateBacklashOffset(self, x, relative=False):
715         w1 = 0# .1185
716         w0 = 0#-.0756
717         if not relative:
718             backlash = w0 + w1*(x - self.origin.x)
719         else:
720             backlash = w0 + w1*(x)
721         return 90+backlash
722
723     #endregion

```

```

724
725     #region Calibration
726
727     #Move to one corner and slowly descend on it
728     def bedCalibrationStep(self):
729         #Move home on first step
730         if self.bedCalStep == 1:
731             self.root.armController.moveHome()
732             self.checkFlag()
733
734         #End calibration
735         if self.bedCalStep >= 5: #Is at 6 after 4 is completed so done
736             self.endSweepOrCal()
737             return
738         #set label
739         currentCornerPos = self.calibrationCorners[self.bedCalStep-1]
740         self.root.cornerLabelHome.config(text=f"Current Corner: {self.bedCalStep}")
741
742         #For better positioning move home than origin so J4 starts at 0 rather than 180
743         #self.root.armController.moveHome()
744         #self.root.armController.moveOrigin()
745
746         #Move above
747         posStep = copy.deepcopy(currentCornerPos)
748         posStep.z = self.plateHeight
749         posStep.z += self.bedCalibrateHeight
750         moveParameters = copy.deepcopy(self.defaultPrintParameters)
751         moveParameters.wrist = "N"
752         self.root.armController.sendMJ(posStep,moveParameters=moveParameters,timeout=self.timeoutExtra)
753         self.checkFlag()
754         #Move halfway
755         self.root.armController.sendMJ(posStep,moveParameters=self.defaultPrintParamete-
rs,timeout=self.timeoutExtra)
756         posStep.z -= self.bedCalibrateHeight/2
757
758         #Move to corner to touch plate
759         self.root.armController.sendMJ(currentCornerPos,moveParameters=self.defaultPrintParamete-
rs,timeout=self.timeoutExtra)
760
761         self.checkFlag()
762         self.root.statusPrint(f"Corner {self.bedCalStep} calibration complete")
763         self.bedCalStep += 1
764
765
766
767
768     # Used to sweep the corners without lifting to ensure kinematics are level to bed
769     def cornerSweep(self, height=None, full=False):
770         self.checkFlag
771         #Height is assumed to be the height of the calibration corners
772         if height is None:
773             height = self.plateHeight
774
775         moveOrder = [2,3,4,1,3,2,4] #1 will always be first
776         moveOrder= [x-1 for x in moveOrder] #adjust to 0 base index

```

```

777
778     pos = copy.deepcopy(self.calibrationCorners[0])
779     pos.z = height
780     self.root.cornerLabelHome.config(text=f"Current Corner: {1}")
781     moveParameters = copy.deepcopy(self.defaultPrintParameters)
782     moveParameters.wrist = "N"
783     #Move to first corner non linearly
784     self.root.armController.sendMJ(pos,moveParameters=moveParameters, timeout=self.timeoutExtra)
785
786     for i in moveOrder:
787         time.sleep(1)
788         pos = copy.deepcopy(self.calibrationCorners[i])
789         pos.z = height
790         self.root.cornerLabelHome.config(text=f"Current Corner: {i+1}")
791         if self.cornerSweeping:
792             self.root.armController.sendML(pos,moveParameters=self.defaultPrintParameters,
793             timeout=self.timeoutExtra)
794             #dont continue if no longer sweeping
795             else:
796                 return
797             self.checkFlag()
798
799     #End calibration
800     #if not doing a fullSweep()
801     if not full:
802         self.endSweepOrCal()
803
804     #End any corner sweep or bed calibration
805     #Moving home is on by default
806     def endSweepOrCal(self, move=True):
807
808         self.cornerSweeping = False
809         self.bedCalibrationInProgress=False
810         self.bedCalStep == 0
811         self.root.cornerLabelHome.config(text=f"Current Corner: N/A")
812         #Wait until move finishes to send move home command
813
814         if move:
815             while self.root.armController.checkIfBusy():
816                 pass
817             self.root.armController.moveHome()
818             self.root.terminalPrint("Ended sweep or cal")
819
820         #use to pause everything going on
821         def pauseAll(self):
822             if self.cornerSweeping or self.bedCalibrationInProgress:
823                 self.endSweepOrCal(move=False) #end sweep without moving
824             if self.printing:
825                 self.pausePrint()
826
827         #sweep multiple layers 20mm at a time
828         def fullCornerSweep(self):
829             self.root.armController.moveHome()
830             self.root.armController.moveOrigin()

```



```
830     heightDelta = 200
831     heightStep = 20
832     currentHeight = self.plateHeight
833     endHeight = currentHeight + heightDelta
834     while currentHeight < endHeight and self.cornerSweeping:
835         self.cornerSweep(height=currentHeight,full=True)
836         currentHeight += heightStep
837         self.checkFlag()
838
```